

Подсистема поиска для доставки еды

Цель: за короткий запрос главной страницы показать 15 доступных поблизости ресторанов, поддержать ограничение радиуса и дать подсказки по названиям ресторанов и блюд.

Сервисы и пользовательские сценарии

Сценарий	Путь запроса	Результат
Главная страница	Клиент → API Gateway → Discovery Service → Redis/H3	Ближайшие доступные рестораны; затем точный расчёт расстояния и сортировка top-15.
Фильтр расстояния	Тот же endpoint /nearby? lat&lon&radius	Discovery выбирает H3-ячейки, пересекающие круг, и отбрасывает кандидатов вне заданного радиуса.
Автодополнение	Клиент (debounce) → Search Suggest Service → Redis/FST	Top-N подсказок для префикса: ресторан или блюдо, с типом и популярностью.

Исходные данные и хранилища

PostgreSQL — источник истины

Транзакционно хранит рестораны, координаты, зоны доставки, меню, доступность и статусы. Это нормализованные изменяемые бизнес-данные, для которых важны ACID-операции. У primary есть read-реплики для гидратации карточек; поиск не сканирует PostgreSQL на критическом пути.

H3 geo-index

Для каждого активного ресторана хранится запись `H3 cell → restaurant_id` на подходящем разрешении. Запрос ближайших кафе начинается с ячейки пользователя и k-ring, расширяя кольца, пока не собраны кандидаты. После этого считается точная геодезическая дистанция и применяется фильтр радиуса. H3 легко шардировать по префиксу ячейки и он подходит для равномерного горизонтального масштабирования.

Prefix index / FST

Индекс содержит нормализованные варианты названий: ресторанов, блюд, алиасов; терминальные записи содержат ID, тип сущности, popularity/rank. FST/компактное префиксное дерево выполняет поиск за $O(|prefix| + N)$ и быстрее таблицы при высокой частоте autocomplete. Индекс read-only между обновлениями, поэтому его можно реплицировать и атомарно переключать версию.

Redis Cluster

Кэширует популярные ключи `nearby:H3:radius:filters` и `suggest:prefix:locale` с коротким TTL. Это уменьшает p95-латентность и нагрузку на индексы; кэш не является источником истины.

Сбор и обработка данных

Изменение в PostgreSQL (меню, название, координаты, открыто/закрыто) в той же транзакции записывается в outbox. CDC публикует событие `catalog.changed` в Kafka. Два consumer worker-а идиempотентно обрабатывают поток:

- **Geo Indexer:** удаляет старую H3-запись и upsert-ит новую; при массовом изменении выполняет reindex.
- **Suggest Indexer:** нормализует текст (lowercase, Unicode, токены, алиасы), пересчитывает веса, строит новую версию FST и делает atomic swap.

Ключ события — `restaurant_id`; это сохраняет порядок изменений одного ресторана. Offset Kafka коммитится только после успешной записи. Для первоначальной загрузки воркеры читают snapshot PostgreSQL, затем догоняют Kafka от сохранённой позиции.

Масштабируемость, надёжность и отзывчивость

- Сервисы stateless: несколько реплик за балансировщиком, autoscaling по QPS и p95. H3 и FST шардируются; Kafka масштабируется партициями и consumer groups.
- PostgreSQL защищён primary + read replicas, бэкапами и PITR. Outbox+CDC не допускают «тихой» потери обновления между БД и индексами. Повторная обработка безопасна за счёт idempotency/version.
- При недоступном индексе сервис отдаёт короткоживущий Redis-кэш или последнюю опубликованную версию индекса и метрику деградации; не возвращает произвольные данные из устаревшего primary.
- Клиент отправляет autocomplete после debounce 150–250 мс, отменяет устаревший запрос и передаёт минимум 2 символа. На сервере: лимит N, таймауты, rate limit, connection pooling.
- Карточки выдаются батчем; включается read-through cache и pre-warm популярных H3-ячеек. Метрики: p50/p95/p99, cache hit, lag Kafka, freshness индексов, доля пустых результатов.